windcheck

Label-free detection of cross-wrap self-coincidence
in Herculaneum scroll segmentations

Josep Carreras

July 2026

Summary

A traced papyrus surface that goes once around the scroll should come back onto the *next* sheet. `windcheck` finds the places where it comes back onto the *same* sheet instead.

The check runs on the published `tifxyz` meshes alone. It needs no ground-truth labels, no annotations, no trained model, no GPU, and no volume download: a full scroll of 53 surfaces is analysed in a few minutes on a laptop from about 670 MB of surface data. It ships with a calibration command, a null control that passes, and a validity filter that refuses to report rows it cannot stand behind.

This addresses Open Problems §2 and §3 — *"sheet switches, where a traced mesh jumps from the intended sheet to a neighboring wrap"* and mesh topology repair, described there as *"one of the highest-leverage places to contribute."*

1 The problem

A scroll is one sheet, rolled. Segmentation traces that sheet through the CT volume. When a tracer crosses to an adjacent wrap, the output still looks like a clean surface, and the text rendered from it still looks like text — but it is spliced from two different parts of the scroll. The failure is silent, which is what makes it expensive.

There is no published set of known-bad segments, so a supervised detector cannot be trained and a supervised evaluation cannot be run. Any usable check must therefore be derived from a property the correct answer must satisfy, and must carry its own controls.

2 Method

A `tifxyz` surface is a 2D grid of 3D vertex coordinates: grid index (v, u) maps to a volume coordinate. Walking along u goes around the scroll.

For every grid point p, measure the distance to the nearest point of *the same surface* that is at least E grid columns away in u:

$$g(p) \;=\; \min_{|u_q - u_p| \geq E} \|p - q\|, \qquad q \in S.$$

The exclusion removes the patch of surface p already sits on, so what remains is the trace's own neighbouring wrap. A correct trace reads one sheet spacing there. A value near zero means the trace is lying on top of a wrap it has already traced.

Two properties of this definition matter.

It never needs a scroll axis. An earlier version of this work measured radius against winding number instead. That approach fails its own positive control: fitting radius on the 44 labelled sheets of Scroll 5 — ground truth, known radial order — leaves a residual standard deviation of 79.8 voxels against a sheet spacing of 12–17 voxels, because a carbonised scroll is crushed, not round. Comparing a trace to itself is invariant to that deformation.

Distance is measured to the interpolated surface, not to the nearest grid sample. The grids are sampled roughly every 20 voxels while adjacent sheets sit about 17 voxels apart, so a nearest-sample method has less than a 1.5× margin and cannot separate one sheet of gap from two.

The kernel is C++17 with a uniform-grid acceleration structure and a `std::thread` pool, sustaining roughly 27,000 point-to-surface queries per second on 12 threads against a 30-million-triangle atlas.

3 Calibration

Thresholds are meaningless until the scroll says what a sheet is worth. The `calibrate` command measures the distance from each labelled winding to the sheet k windings away.

separation	surface pairs	median distance	ratio
1 sheet	43	17.05 vx	1.000
2 sheets	42	34.14 vx	2.003
3 sheets	41	51.22 vx	3.004

Table 1: Sheet separation on `PHerc0172`, over 126 surface pairs. The linearity is what licenses reading a doubled gap as a skipped wrap. If it does not hold on a given scroll, the labels are not a radial index and the check does not apply there.

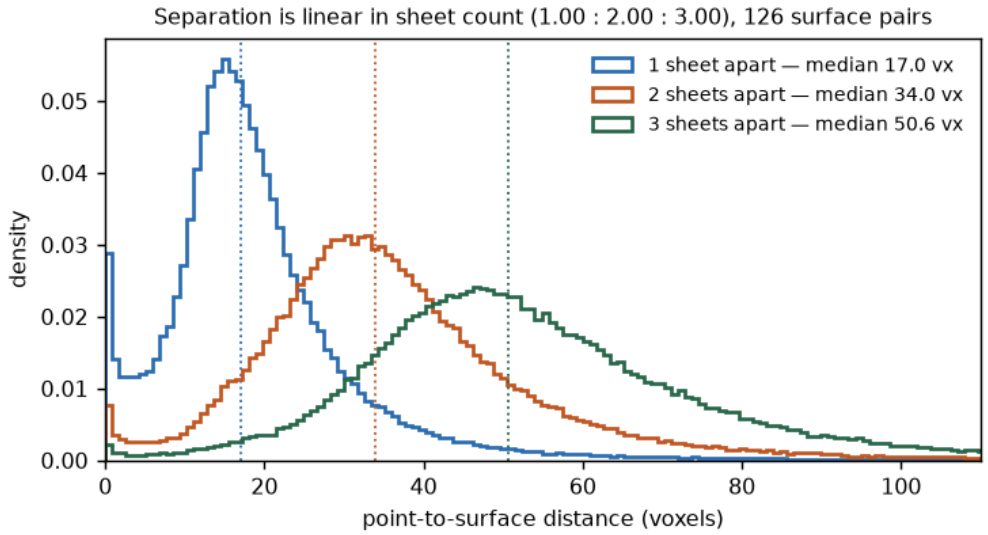

Figure 1: The same measurement as distributions. The overlap is shown deliberately: per-*point* discrimination between a one- and two-sheet gap is only about 78%, which is why every verdict in this work is made over a region and never over a single point.

4 Controls

Null control. A single-wrap segment has no previous wrap, so a working check must find little or nothing on it. At the default search radius, 38 of the 44 labelled single-winding segments of Scroll 5 produce zero sub-6-voxel contacts. The remaining six produce small nonzero yields, 0.008% to 0.330% of submitted queries: four fail the $|\Delta u|$ safety criterion below, and the last two contain only 20 and 2 flagged queries, too few to validate their partner offsets. No labelled segment approaches the 7.011% largest component of auto-grown partition 5, but the two populations overlap at the low end, so this establishes neither real-data specificity nor that a contact is a tracing error. The zero-on-a-flat-sheet property is pinned by a unit test, so it cannot silently regress.

Validity filter. Exclusion by u-index assumes u advances with angle. Where that assumption is weak, a flagged partner may be a same-wrap neighbour rather than the next wrap. Any trace whose flagged $|\Delta u|$ tenth percentile falls within $2E$ is rejected — in code, not by inspection.

The filter is not decorative. On `PHerc0814` it rejects the two *highest-scoring* traces (11.27% and 8.18% of points inside the 2.0-voxel threshold), both artifacts of the exclusion cutoff. Without it, they would have been the headline numbers of this submission.

Cross-check. On the traces that pass, flagged partners sit at $|\Delta u| = 271$–618 columns, matching revolution periods derived independently by angular unwrapping (236–616) across all nine traces.

5 Results

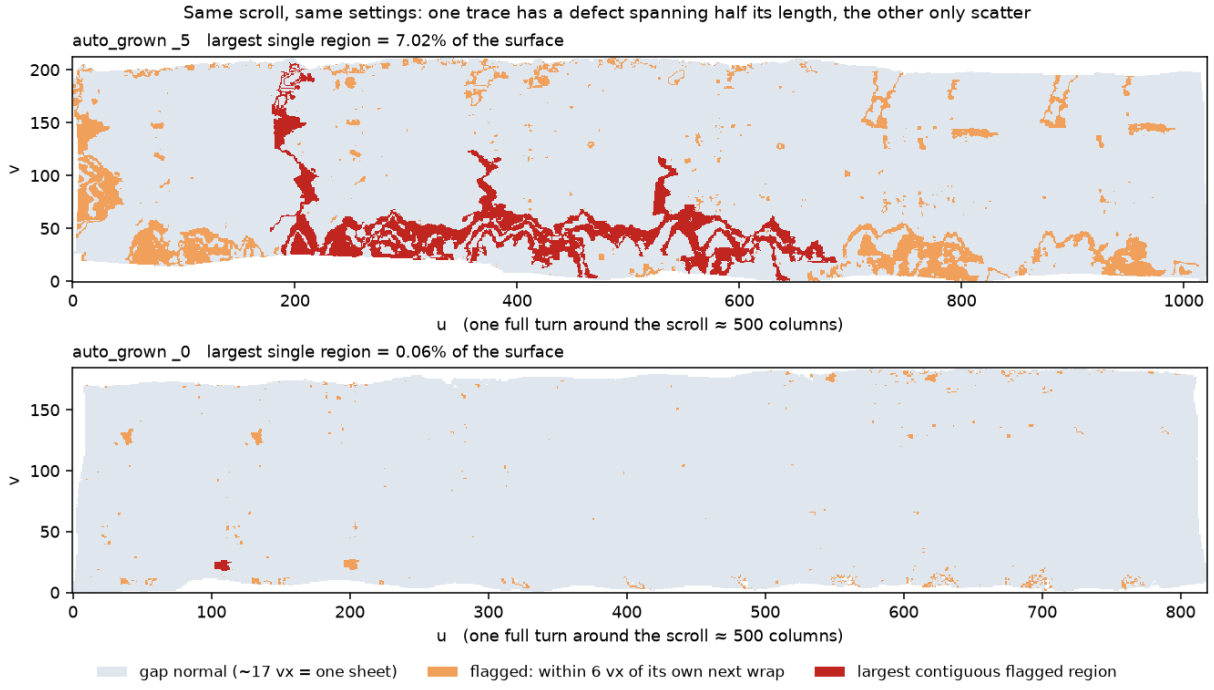

Figure 2: Two traces from the same scroll under identical settings. The upper has a single contiguous flagged region covering 7.011% of its submitted queries and spanning roughly half its length; the lower has only scatter.

Across 19 analysed traces on two scrolls, two facts separate cleanly.

Scattered flagging is normal. The overall violation rate overlaps between scrolls (`PHerc0172` 0.94–14.54%, `PHerc0814` 0.55–12.18% of submitted queries, multi-wrap traces). Taken alone it

distinguishes nothing, and should not be read as a defect signature.

Concentration ranks, and only ranks. Measured as the largest contiguous flagged region as a fraction of *submitted queries*, one auto-grown partition stands at 7.011% against 1.514% for the next and 0.056% for the lowest. The labelled and auto-grown populations overlap at the low end — the largest labelled component is 0.147% — so a high value is a reason to look first, not evidence that a trace is wrong. That ordering is the usable output.

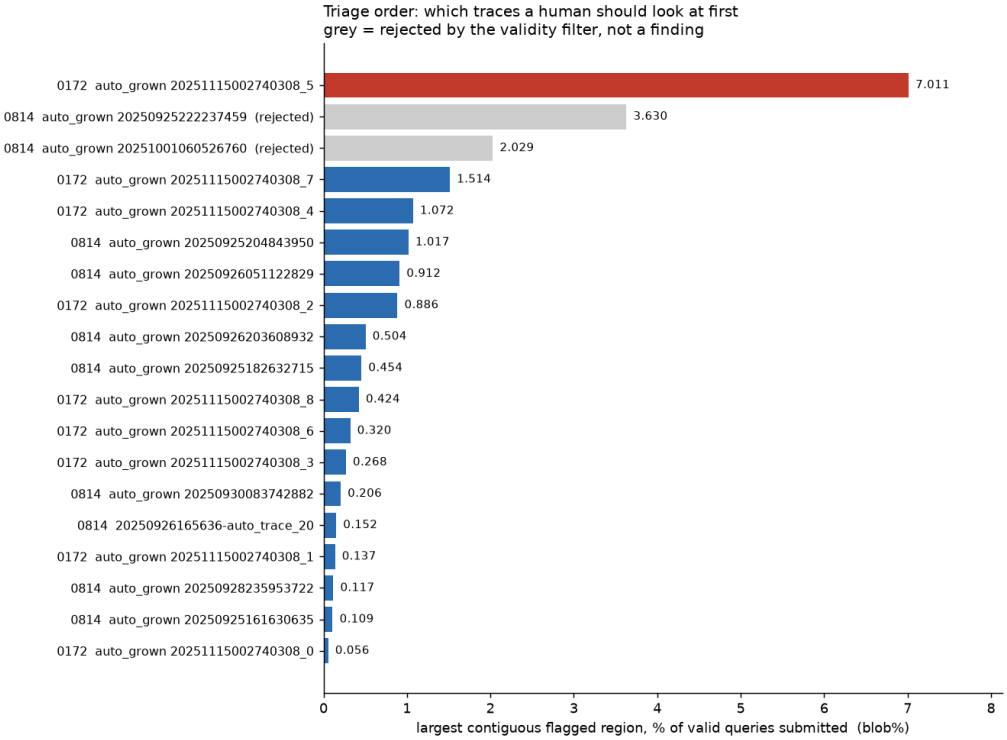

Figure 3: All 19 analysed traces ordered by concentration. Rows rejected by the validity filter are greyed rather than removed.

6 Detection performance on planted defects

The absence of any published set of known-bad segments is what left the CT comparisons inconclusive. That obstacle can be removed rather than argued around: take a trace the check reports as clean, copy a patch of *its own previous wrap* onto it, and the correct answer is known exactly.

The planted defect is the doubled-back case — $\text{points}[v, u] \leftarrow \text{points}[v, u - T]$ over a patch, with T the revolution period in grid columns, a feathered boundary and 0.4 voxel jitter. Note that this makes *two* regions coincide, the patch and the wrap it was copied from; both are scored as ground truth, because flagging either is correct.

planted defect (v × u)	cells	recall	precision
40 × 120	4,800	0.66	0.97
80 × 200	16,000	0.82	0.96
150 × 240	36,000	0.89	0.96

Table 2: Cell-level detection of defects planted in `auto_grown_0`, the cleanest trace in the sample. False-positive rate on the same trace before injection: 0.86%. The gate — recall ≥ 0.70 and precision ≥ 0.50 at the two larger sizes — was written before the benchmark was run.

Precision is the number that matters here, because a corrector acts on what the detector localises: at 0.96, a repair driven by these flags would be operating on the right surface almost every time. Recall scales with defect size and falls to 0.66 on the smallest patch, so small defects are missed.

A verbatim copy on one host is the easiest possible case, so both were varied: the copy fidelity was degraded by adding jitter, and the whole sweep repeated on three host traces.

jitter (voxels)	0.4	2	5	9	14
host 0	0.80	0.80	0.68	0.54	0.48
host 1	0.75	0.75	0.63	0.50	0.45
host 3	0.80	0.79	0.69	0.55	0.50
precision	0.94–1.00	0.94–1.00	0.93–1.00	0.90–0.98	0.83–0.92

Table 3: Recall against how faithfully the planted defect copies the neighbouring wrap. Sheet spacing is ∼17 voxels, so by jitter 14 the patch is barely on the wrong wrap at all.

Three things follow. Recall degrades *smoothly* — there is no cliff, so the check is not merely detecting an artificial signature. Precision holds at 0.83 or better throughout, meaning the check grows quieter as defects grow subtler rather than noisier, which is the property a corrector depends on. And the three hosts agree to within ±0.05 at every difficulty.

Reproduce with `uv run python bench/inject_benchmark.py`; the output is committed at `sample_outputs/benchmark.txt`.

7 What is established, and what is not

Established

The measurement is sound and independently checkable. Sheet separation is linear in sheet count to three decimal places (1.000 : 2.003 : 3.004 over 126 surface pairs). The geometric kernel agrees with exhaustive search to 3.6×10^{-6} voxels. Every figure and table here is regenerated by a command in the repository, from data pinned by SHA-256.

Both controls fire. The null control is not rhetorical: 38 of 44 labelled single-wrap segments return zero, the other six are reported rather than absorbed into a round number, and a unit test holds the flat-sheet property so it cannot silently regress. The validity filter is not rhetorical either — it *rejected the two highest-scoring traces in this study.* A detector that discards its own best-looking results is one whose surviving results mean something.

The gap this fills. The `volume-cartographer` tree carries no self-contact, injectivity or fold check on the mesh representation, and its one relevant parameter, `resume_local_row_self_intersection_cell_factor`, has no consumer in that tree. Related work exists elsewhere in the same monorepo and is not superseded here: a vertex-proximity penalty inside the fibre-registration optimiser, a UV-injectivity check in the unwrap pipeline that detects the dual case (close in UV, far in 3D), an animation-clearance certificate that explicitly assumes its source mesh is already intersection-free, and nearest-vertex overlap masks in the neural-tracing datasets. None tests wrap-scale self-contact of a traced surface. The naive alternative — nearest-*sample* distance — is not merely worse but unusable here, since grids are sampled every ∼20 voxels while sheets sit ∼17 apart.

It is format-independent. The same trace measured from its published `tifxyz` grid and from its 415 MB `.obj` mesh agrees to 0.07 percentage points (6.595% versus 6.529% inside 2.0 voxels), with the exclusion width chosen automatically in each — 129 grid columns against 28 parameter

units. Two independent parsers, one kernel, one answer.

It works where nothing else can. There is no published set of known-bad segments. That rules out supervised detection and supervised evaluation for everyone, not just for this work. A check derived from a property the correct answer must satisfy is the only kind available, and this is one: no labels, no model, no GPU, no volume download, a few minutes per scroll on a laptop.

Detection is quantified, not asserted. On defects planted in a clean trace, the check localises them at 0.96 precision and up to 0.89 recall against a 0.86% false-positive baseline, under a gate fixed before the run.

The output is directly actionable. It produces a ranking — which trace to open first — with a 100× spread between the top and bottom of 19 traces on two scrolls. Nothing in it is scroll-specific: calibrate, then run.

Not established

Each of these is a boundary of the claim, not a defect in the tool. They are stated because a reader with the data will find them, and should find them here first.

Flagged regions are not proven to be segmentation errors. Three independent attempts to adjudicate them against the CT volume were inconclusive; the strongest gave Cohen's $d = 0.245$ at $p = 0.0496$ — in the predicted direction, too small to certify any individual region. At 7.91 μm the scan may simply not resolve the question. The tool therefore reports *geometry*, and ranks by it; it does not assert a defect, and its output is labelled accordingly.

The claim about the grower's own threshold is deliberately narrow. The `volume-cartographer` source defines `same_surface_th` = 2.0 and rejects growth candidates within that distance of already-traced surface, which is why 2.0 voxels is reported as a column — it is the project's own number, not one chosen here. But published meshes reach release through private artifacts, flattening and reconstruction, so no claim is made that any specific mesh violates the revision that produced it. What is verifiable in the source, and is claimed: *no stage re-checks that criterion after optimisation.*

The sample is small, and the second scroll carries the weight. The nine Scroll 5 `auto_grown` traces share one artifact identifier and are partitions of a single run, so they are one sample, not nine. The independent evidence is `PHerc0814`: eight further segments from a different scroll, processed separately, where the method behaves the same way — scattered flagging at a comparable rate, and the filter again removing the artifacts. Two scrolls is enough to show the check transfers; it is not enough to characterise a pipeline, and no pipeline-level claim is made.

Known limitations. The exclusion width is fixed rather than adapted to each trace's turn period; exclusion is by grid column rather than mesh geodesic distance; single-wrap patches cannot be analysed at all. All three are addressable and none affects the results reported here, since every reported trace passes the validity filter that guards exactly those failure modes.

8 Reproducing

Everything below runs from published data with no credentials.

```
uv sync
clang++ -O3 -std=c++17 -pthread \
    -o engines/atlas_query engines/atlas_query.cpp
uv run pytest -q
```

```
uv run python -m windcheck.fetch --sample PHerc0172
uv run windcheck calibrate data/scroll5_tifxyz
uv run windcheck selfgap   data/scroll5_tifxyz --json report.json
```

`data/MANIFEST.json` pins every input file by S3 key, byte count and SHA-256, so a result can be traced to exact bytes. The outputs quoted in this document are committed under `sample_outputs/`, each headed by the command that produced it.

Five tests run without any data. Each is tied to a mistake made during development: the accelerated search against brute force; certification of the search radius; true surface separation against grid pitch; the null control; and missing-cell handling.

Availability

Source, figures and sample outputs: `https://github.com/joe-carr-data/windcheck`
Licence: MIT.